



Autores:	Data de Criação: 24/10/2025
Suyanne Sara Miranda Silva - 222006445	Última Atualização: 29/10/2025

Documento de Requisitos a Nível de Sistema

INFORMAÇÕES SOBRE O PROJETO

Este é um documento referente ao projeto da matéria de Engenharia de Software (CIC0105), que tem como foco a criação de um produto para auxiliar na aplicação da metodologia Scrum no processo de desenvolvimento de software denominado “Sistema Scrum”.

INFORMAÇÕES SOBRE ESSE DOCUMENTO

O presente documento, System Wide Requirements (SWR), descreve os requisitos a nível de sistema do projeto desenvolvido, enfocando especificamente os requisitos não funcionais do sistema.

Este artefato define os atributos de qualidade, restrições técnicas e exigências de ambiente que garantem o funcionamento adequado do sistema em termos de desempenho, segurança, usabilidade, manutenibilidade e portabilidade.

Quadro 01 - Nome dos Integrantes do Grupo e Matrícula

Nome	Matrícula
Gabriel Francisco de Oliveira Castro	202066571
Lucas Issamu Hashimoto	231003504
Pedro Vale de Souza	231038733
Matheus Duarte da Silva	211062277
Suyanne Sara Miranda Silva	222006445

Fonte: Do Autor

HISTÓRICO DE REVISÃO

Esse trecho contém todo o histórico de revisão e mudanças nesse documento, para garantir um controle e um rastreo da evolução do documento.

Quadro 02 - Histórico de Revisão

Data	Autor	Descrição	Versão
24/10/2025	Suyanne Sara Miranda Silva	Criação do documento	v0.1.0
29/10/2025	Suyanne Sara Miranda Silva	Ajuste na estrutura do documento	v0.2.0



07/11/2025	Suyanne Sara Miranda Silva	Atualização de informações	v1.0.0
13/11/2025	Suyanne Sara Miranda Silva	Atualização de informações	

Fonte: Do Autor

1. REQUISITOS FUNCIONAIS

Além dos requisitos funcionais identificados a partir das histórias de usuários, o sistema deverá atender aos seguintes requisitos funcionais de **caráter sistêmico**, necessários para o pleno funcionamento da plataforma e manutenção da integridade das operações:

- O sistema deve autenticar usuários mediante verificação de CPF e senha cadastrados, permitindo o acesso apenas quando as credenciais corresponderem aos registros do banco de dados.
- O sistema deve manter sessões ativas exclusivamente para usuários autenticados, garantindo que cada sessão possua um identificador único e válido (Bearer Token).
- O sistema deve encerrar automaticamente sessões inativas após 60 minutos consecutivos de inatividade, exigindo novo login para restabelecer o acesso.
- O sistema deve validar os campos de login antes do envio, impedindo o envio de formulários com campos obrigatórios vazios ou com formato inválido.
- O sistema deve permitir a recuperação de senha por meio de envio de link de redefinição para o e-mail cadastrado, garantindo que apenas o titular da conta consiga redefini-la.

2. ATRIBUTOS DE QUALIDADE REQUERIDOS

2.1 Usabilidade

- Todas as imagens e ícones devem conter textos alternativos (alt) descritivos para garantir acessibilidade.
- As mensagens de erro devem seguir um padrão de cor vermelha, indicando claramente a causa do erro e a ação recomendada ao usuário.
- O sistema deve permitir aprendizado rápido, de modo que um usuário familiarizado com metodologias ágeis consiga utilizá-lo intuitivamente em até 10 minutos de uso.
- Deve haver feedback visual de no máximo 100 milissegundos para cada ação executada (por exemplo, botões mudando de estado).

2.2 Confiabilidade e Segurança

- Em caso de falha no backend, a aplicação deve exibir mensagem de erro amigável.
- As senhas dos usuários e as senhas de ingresso aos projetos devem ser criptografadas antes de serem armazenadas no banco de dados.
- A autenticação deve ocorrer por meio de JWT (JSON Web Token), garantindo a integridade das sessões de usuário.
- A comunicação entre o frontend e o backend deve ocorrer exclusivamente via HTTPS (Protocolo de Transferência de Hipertexto Seguro).
- Os tokens JWT devem possuir tempo de expiração configurável e ser invalidados em caso de logout.

2.3 Desempenho

- O tempo de resposta da API deve ser inferior a 2 segundos para todas as requisições.
- O carregamento inicial da interface web (Nuxt.js) não deve ultrapassar 5 segundos em conexões de 10 Mbps.
- O sistema deve suportar no mínimo 50 usuários simultâneos sem degradação perceptível de desempenho.
- O sistema deve permitir cache de páginas estáticas para otimizar o carregamento.

2.4 Suportabilidade e Manutenibilidade

- O sistema deve possuir testes unitários e de integração com cobertura mínima de 80%.
- O processo de deploy deve ser automatizado via GitHub Actions (Continuous Integration), executando testes e build a cada push.
- Deve existir um arquivo .env para variáveis de ambiente, não versionado no repositório.
- O sistema deve registrar logs de erros e eventos relevantes do backend em arquivo local e/ou serviço externo para fins de diagnóstico e manutenção.
- O sistema deve adotar boas práticas de versionamento semântico (Semantic Versioning) para controle de versões e rastreabilidade de atualizações.
- O processo de manutenção deve ser documentado, incluindo instruções para configuração do ambiente, execução de testes e procedimentos de deploy.

3. INTERFACES DE SISTEMA REQUERIDAS

3.1 Interfaces de Usuário

- A interface web deve ser acessível exclusivamente em dispositivos desktop e notebooks, por meio de navegadores modernos (Chrome, Firefox e Opera).
- O sistema deve possuir uma interface coerente e padronizada, com cabeçalho fixo, menu lateral, área principal de conteúdo, estilos de fonte, cores, componentes, formulários e inputs consistentes entre as telas, conforme definido no design de telas por meio da ferramenta Figma.
- O idioma padrão da interface deve ser português (Brasil).
- O sistema deve adotar um design monocromático, utilizando primordialmente tons de preto, branco e cinza.

3.2 Interfaces de Software

- O backend deve expor *endpoints* para autenticação, cadastro de usuários além de CRUDs (Create, Read, Update e Delete) para gerenciamento de projetos, backlogs, sprints e tarefas no kanban.
- O frontend deve se comunicar com o backend exclusivamente por meio de *endpoints*, utilizando o formato JSON para requisições e respostas.
- A API deve seguir convenções REST, utilizando métodos e códigos de resposta padrão (GET, POST, PUT, DELETE, 200, 404, 500, etc.).

3.3 Interfaces com Dispositivos Externos

- O sistema não deve possuir integração direta com dispositivos externos, como sensores, impressoras fiscais, leitores de código de barras, catracas ou dispositivos móveis especializados.

3.4 Interfaces do Sistema com Outros Sistemas

- O sistema não deve possuir integração direta com sistemas externos de gerenciamento, monitoramento ou plataformas de terceiros, exceto pelo uso do protocolo OAuth2 integrado ao backend para fins de autenticação e autorização de usuários.

4. ASPECTOS DE CONFORMIDADE

- O sistema deve estar em conformidade com a Lei nº 13.709/2018, Lei Geral de Proteção de Dados Pessoais (LGPD), garantindo a privacidade e segurança das informações dos usuários.
- Os registros de logs devem ser mantidos por período mínimo de 180 dias e máximo de 1 ano, sendo acessíveis apenas a administradores autorizados. Após o prazo de retenção, os logs devem ser automaticamente excluídos de forma segura.



5. RESTRIÇÕES DE PROJETO (DESIGN)

- O backend deve ser implementado em Python 3.10+, utilizando SQLite como banco de dados e SQLAlchemy CORE para reflexão das tabelas..
- O frontend deve ser desenvolvido em Nuxt.js 3+, empregando Axios para comunicação HTTP com a API backend.
- O processo de deploy deve ser automatizado via GitHub Actions, incluindo etapas de build, teste e publicação contínuas (CI/CD).
- O código-fonte deve ser armazenado e versionado em repositório remoto GitHub, garantindo rastreabilidade de alterações.
- O sistema deve ser compatível com navegadores modernos (Chrome, Firefox, Opera) e padrões web atuais (HTML5, CSS3, ECMAScript 6+).
- As dependências do projeto devem ser gerenciadas por ferramentas oficiais das respectivas plataformas, pip para *backend* e npm para *frontend*.

6. ASPECTOS DE LICENCIAMENTO

- O sistema deve utilizar somente bibliotecas e dependências de código aberto compatíveis com licenças permissivas (ex.: MIT, Apache 2.0, BSD).
- As licenças das dependências do sistema devem ser registradas no repositório do projeto e revisadas mensalmente para assegurar conformidade com seus termos de uso.
- O código-fonte do sistema deve ser armazenado em repositório remoto sob licença MIT.

7. DOCUMENTAÇÃO REQUERIDA

- O sistema deve possuir documentação técnica contendo instruções de instalação, configuração, deploy e manutenção.
- Devem ser produzidos diagramas UML (caso de uso, classes, sequência e atividades) representando a arquitetura e os fluxos principais do sistema.
- O repositório deve conter um arquivo README.md com informações gerais sobre o projeto, dependências e instruções de execução.
- Deve ser mantido um manual do usuário descrevendo as principais funcionalidades e fluxos da aplicação.



Universidade de Brasília
Instituto de Ciências Exatas
Departamento de Ciências da Computação

- A documentação de código deve seguir padrões de docstring e comentários compatíveis com as convenções PEP 257 (Python).
- As instruções de ambiente e variáveis sensíveis devem ser especificadas em um arquivo *.env.example*, sem expor informações sigilosas.